PES-VCS Project Report

Name : Ritesh Babu Reddy
Sec : G
SRN : PES2UG24CS403

Date: 21-4-2026

Github Link: https://github.com/RiteshReddyy/PES2UG24CS403-pes-vcs

PHASE -1

1)A)

```
ritesh@LAPTOP-6HJONCNH:~/PES2UG24CS403-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1)B)

```
ritesh@LAPTOP-6HJONCNH:~/PES2UG24CS403-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

PHASE-2

2)A

```
ritesh@LAPTOP-6HJONCNH:~/PES2UG24CS403-pes-vcs$ make test_tree && ./test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c: In function 'write_tree_level':
tree.c:123:13: warning: '__builtin___strncpy_chk' output may be truncated copying 255 bytes from a string of length 255 [-Wstringop-truncation]
  123 |             strncpy(te->name, subdir_name, sizeof(te->name) - 1);
      |             ^
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2)B

```
ritesh@LAPTOP-6HJONCNH:~/PES2UG24CS403-pes-vcs$ make pes
./pes init
echo "hello" > file1.txt
./pes add file1.txt
find .pes/objects -type f | head -1 | xargs xxd | head -20
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
```

PHASE 3

3)A

```
ritesh@LAPTOP-6HJONCNH:~/PES2UG24CS403-pes-vcs$ ./pes status
Staged changes:
    staged: bye.txt
    staged: file1.txt
    staged: file2.txt

Unstaged changes:
    (nothing to show)

Untracked files:
    untracked: pes.h
    untracked: tree.h
    untracked: hello.txt
    untracked: index.c
    untracked: test_tree.c
    untracked: object.c
    untracked: debug_add
    untracked: .gitignore
    untracked: commit.c
    untracked: index.h
    untracked: commit.h
    untracked: Makefile
    untracked: debug_add.c
    untracked: tree.c
    untracked: README.md
    untracked: test_sequence.sh
    untracked: test_objects.c
    untracked: pes.c
```

3)B)

```
ritesh@LAPTOP-6HJONCNH:~/PES2UG24CS403-pes-vcs$ cat .pes/index
100644 5c4e1e2f5a561dd416ca159203c3259111f64259b7341e2754484f9b8209b0ae 1776754094
4 bye.txt
100644 cab4edff685f6b5cd7197f73179839ca37a821be03a84e0fa10a82d74c12ab14 1776754093
18 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776754093
6 file2.txt
```

PHASE 4

4)A)

```
ritesh@LAPTOP-6HJONCNH:~/PES2UG24CS403-pes-vcs$ ./pes log
commit a81e7a26d4e90eb0392a88ddc58c014d44869ed73cea853c230c7c2944d2aeb5
Author: PES User <pes@localhost>
Date:   1776754094

    Add bye

commit aa7815bb8397004957f24cd0c9376122501f3a20e2f95212298095fdbabf87bb
Author: PES User <pes@localhost>
Date:   1776754094

    Update file1

commit 00ce1f0594c55da5f521a4de00fa2b570cc442db53e7ef90dbf93b59b4554cb5
Author: PES User <pes@localhost>
Date:   1776754093

    Initial commit

commit 9102312994272a2508251862bf4ea11e57b25b24024ccf31ec1e784f85154bbe
Author: PES User <pes@localhost>
Date:   1776754037

    Add bye

commit ff32cf1ec94930caa5a0580b38fd37952b329ea051d259537f4fed59db955a99
Author: PES User <pes@localhost>
Date:   1776754037

    Update file1

commit 7a3b17edb634cee20a679631c28fccfe9addc39493a53b8cd47098883a6d8a69
Author: PES User <pes@localhost>
Date:   1776754037

    Initial commit
```

4)B)

```
ritesh@LAPTOP-6HJONCNH:~/PES2UG24CS403-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/00/ce1f0594c55da5f521a4de00fa2b570cc442db53e7ef90dbf93b59b4554cb5
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/5c/4e1e2f5a561dd416ca159203c3259111f64259b7341e2754484f9b8209b0ae
.pes/objects/6d/b08a35674661c57125b8a1d15432ddd199d6938425ea3c08ca397a5b3a3493
.pes/objects/7a/3b17edb634cee20a679631c28fccfe9addc39493a53b8cd47098883a6d8a69
.pes/objects/91/02312994272a2508251862bf4ea11e57b25b24024ccf31ec1e784f85154bbe
.pes/objects/a8/1e7a26d4e90eb0392a88ddc58c014d44869ed73cea853c230c7c2944d2aeb5
.pes/objects/aa/7815bb8397004957f24cd0c9376122501f3a20e2f95212298095fdbabf87bb
.pes/objects/b1/514880953876436ac11d1762cd3bb8303bbc70018595a28ebb329c1e23a258
.pes/objects/ca/b4edff685f6b5cd7197f73179839ca37a821be03a84e0fa10a82d74c12ab14
.pes/objects/de/62c80dfbd1163d8b4ee3273c30129e26225ec5521afcf599594b0e2959611d
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/f5/902ee73b6aa25436f687067fc4cc0cb481cf416740d3c3c3ed6d9c4de7b6c2
.pes/objects/fe/76325aa5521b207ebe01e12fd8e9e3abf030cacd5398e3744a3a56a81ad1bd
.pes/objects/ff/32cf1ec94930caa5a0580b38fd37952b329ea051d259537f4fed59db955a99
.pes/refs/heads/main
```

4)C)

```
ritesh@LAPTOP-6HJONCNH:~/PES2UG24CS403-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
a81e7a26d4e90eb0392a88ddc58c014d44869ed73cea853c230c7c2944d2aeb5
ref: refs/heads/main
```

PHASE -5

5)A) To implement `pes checkout <branch>`, three things need to change in `.pes/`: HEAD must be updated to point to the new branch (e.g., `ref: refs/heads/<branch>`), and the working directory files must be updated to match the target branch's tree. The process is: read the target branch's commit hash from `.pes/refs/heads/<branch>`, read that commit object to get its tree hash, recursively walk the tree object to get all file paths and their blob hashes, then for each file read the blob from the object store and overwrite the working directory file. What makes this complex is handling files that exist in the current branch but not the target (they must be deleted), files that exist in the target but not current (they must be created), and conflicts where the user has local changes that would be overwritten.

---

5)B) To detect a dirty working directory conflict, for each file in the current index: compute its current blob hash by reading the file from disk, then compare it to the hash stored in the index. If they differ, the file has been modified since it was last staged. Additionally, compare the index entries against the target branch's tree — if a file differs between the two branches AND has local modifications, checkout must refuse. This requires no extra data: just the index (which stores the last-staged hash) and the object store (which stores the target branch's version).

---

5)C) In detached HEAD state, HEAD contains a raw commit hash instead of `ref: refs/heads/<branch>`. New commits are still created and chained correctly, but no branch pointer is updated to track them. If you switch away, those commits become unreachable — no branch points to them. To recover, you would need the commit hash (visible in terminal history or `pes log` output) and then create a new branch pointing at that commit, making it reachable again before garbage collection removes it.

PHASE - 6

6)A) The algorithm is mark-and-sweep. Start from all branch refs in `.pes/refs/heads/` and HEAD, then walk every commit in every branch's history. For each commit, mark its hash as reachable, then mark its tree hash, then recursively mark all blobs and subtrees that tree points to. Any object file in `.pes/objects/` whose hash is not in the reachable set can be safely deleted. A hash set is the right data structure for tracking reachable hashes. For 100,000 commits across 50 branches with an average of 10 files per commit, you would visit roughly 100,000 commits + 100,000 trees + 1,000,000 blobs = about 1.1 million objects total.

---

6)B) The race condition is: GC scans the object store and finds a blob that no commit currently references, so it marks it for deletion. Simultaneously, a `pes add` writes that same blob to the object store and a `pes commit` is about to reference it in a new tree and commit object. If GC deletes the blob between the `add` and the `commit`, the new commit will reference a blob that no longer exists, corrupting the repository. Git avoids this by using a grace period — objects newer than a certain age (default 2 weeks) are never deleted by GC, giving any in-progress operations time to complete and create references to new objects before GC can touch them.